

RL-Based Portfolio Budgeting: Complete Paper Strategy & Architecture

1. Core Problem & Motivation

1.1 The Challenge

Portfolio budgeting under uncertainty faces three critical barriers:

- **Computational Intractability:** Classical OR (MIP, Stochastic Programming) requires minutes to hours per instance, making real-time decisions and large-scale scenario analysis infeasible.
- **Scalability Failure:** MIP solve time grows exponentially with portfolio size; re-optimization after budget changes requires full re-solve.
- **Data Limitations:** Real-world project data is confidential; regional economic distortions (inflation, sanctions) invalidate direct use for international publication.

1.2 The Opportunity

A trained RL agent, once deployed, produces allocation plans in milliseconds — enabling use cases that classical OR simply cannot support at scale.

2. Core Contributions (Ordered by Priority)

Contribution 1 — Computational Speedup (Primary, Must-Have)

"Our RL agent achieves a 10,000× speedup over classical MIP solvers (100ms vs. 15–45 min/instance), enabling real-time decision support and large-scale scenario analysis previously infeasible with traditional OR methods."

Why this matters:

- Portfolio manager evaluating 1,000 scenarios → MIP: ~250 hours, RL: ~2 minutes
- Urgent budget reallocation → MIP: wait 30 min, RL: instant response
- Monte Carlo simulation over uncertainty → MIP: infeasible, RL: trivial

How you prove it:

- Table: solve time vs. portfolio size (N = 10, 20, 50, 100, 200 projects)
- Show MIP time grows exponentially; RL inference stays flat (~100ms)
- Highlight practical scenario: "1,000 budget scenarios evaluated in under 2 minutes"

Contribution 2 — Solution Quality Under Uncertainty (Primary, Must-Have)

"The RL agent outperforms deterministic MIP by ~18% in expected portfolio return and matches stochastic programming performance — without requiring explicit scenario enumeration — achieving ~92% of the perfect-information upper bound."

Why this matters:

- Deterministic MIP ignores uncertainty → poor real-world performance
- Stochastic programming requires manual scenario definition → expert bottleneck
- RL learns robust policies directly from stochastic simulation

How you prove it:

- Comparison table: RL vs. Greedy vs. MIP vs. Stochastic Programming
- Metric: expected return, variance, budget utilization rate
- Perfect-information MIP as upper bound → show RL gap is small (~8%)
- Sensitivity analysis: vary uncertainty level (low/medium/high) and show RL stays robust

Contribution 3 — Open-Source Deployable Framework (Secondary, High-Impact)

"We release an open-source decision support system with full backend API and interactive frontend, enabling practitioners to deploy portfolio optimization without ML expertise. For industry-specific adaptation, we provide a fine-tuning protocol"

using historical project data, bridging the gap between synthetic training and real-world deployment."

Why this matters:

- Most OR papers stop at experimental results — you ship working software
- Addresses the synthetic data limitation directly: pretrain on synthetic, fine-tune on real
- Democratizes access: no OR expert or ML engineer needed for deployment
- Tangible impact: reviewers can actually use your system

What you build:

Component	Purpose	User Experience
Backend API	RL inference engine	REST endpoints for allocation requests
Frontend UI	Interactive dashboard	Upload projects, set budget, get allocation plan
Fine-tuning module	Domain adaptation	Company uploads historical data, system retrains
Documentation	Deployment guide	Step-by-step setup for practitioners

How you prove it:

- Demo video or screenshots in supplementary material
- GitHub repository with full codebase
- Fine-tuning case study: synthetic "company dataset" → show performance improvement
- Deployment guide: "from zero to running system in 30 minutes"

The fine-tuning angle specifically:

- Pretrain on 10,000 synthetic instances (broad knowledge)
- Fine-tune on 200-500 company-specific instances (domain specialization)
- Show: fine-tuned model outperforms generic model by 8-12% on company data
- This is exactly the LLM paradigm — reviewers in 2025-2026 will immediately get it

Strategic value:

- Turns synthetic data from weakness to strength: "we use synthetic for pretraining, real data for fine-tuning"
- Opens a natural future work section: "how much real data is needed for effective fine-tuning?"
- For PhD applications: you can demo a live system, not just show tables

Contribution 4 — Interpretability Analysis (Optional, Add If Time Permits)

"The RL policy provides interpretable allocation signals through attention-based analysis, revealing which project features drive budget decisions."

Why this matters for OR journals:

- Practitioners don't trust black boxes
- Reviewers at EJOR/Omega want auditability
- Adds a "managerial insights" section with real substance

Implementation options (pick one):

Option	Effort	Output
Attention mechanism in policy network	1 week	Heatmap of project feature focus
SHAP value analysis post-training	3 days	Feature importance ranking
Comparative RL vs. MIP decision analysis	1 week	"RL agrees with MIP on 85% of top projects"
Policy distillation to decision tree	2 weeks	Human-readable allocation rules

Recommendation: Go with SHAP or comparative analysis — lowest effort, publishable output.

3. What You Build

3.1 Synthetic Environment (Your Data Foundation)

- Generate 10,000 portfolio instances
- Parameters calibrated from international OR/PPM literature
- Each instance: N projects, budget constraint B, stochastic returns/costs/risks
- Validate distributions against literature statistics
- Publish full parameter spec as JSON (reproducibility)

Why synthetic data is acceptable:

- Real data is confidential and regionally distorted
- Literature-calibrated synthetic data is standard practice in OR computation papers
- You explicitly acknowledge and justify this in Section 4.4.1
- Fine-tuning protocol addresses real-world deployment

3.2 RL Agent

- Algorithm: PPO (stable, well-understood, easy to tune)
- State: current budget remaining, project feature vectors, portfolio state
- Action: budget allocation vector across N projects
- Reward: expected portfolio return under uncertainty (penalize constraint violations)
- Architecture: MLP or attention-based policy network

3.3 Baselines (All Required)

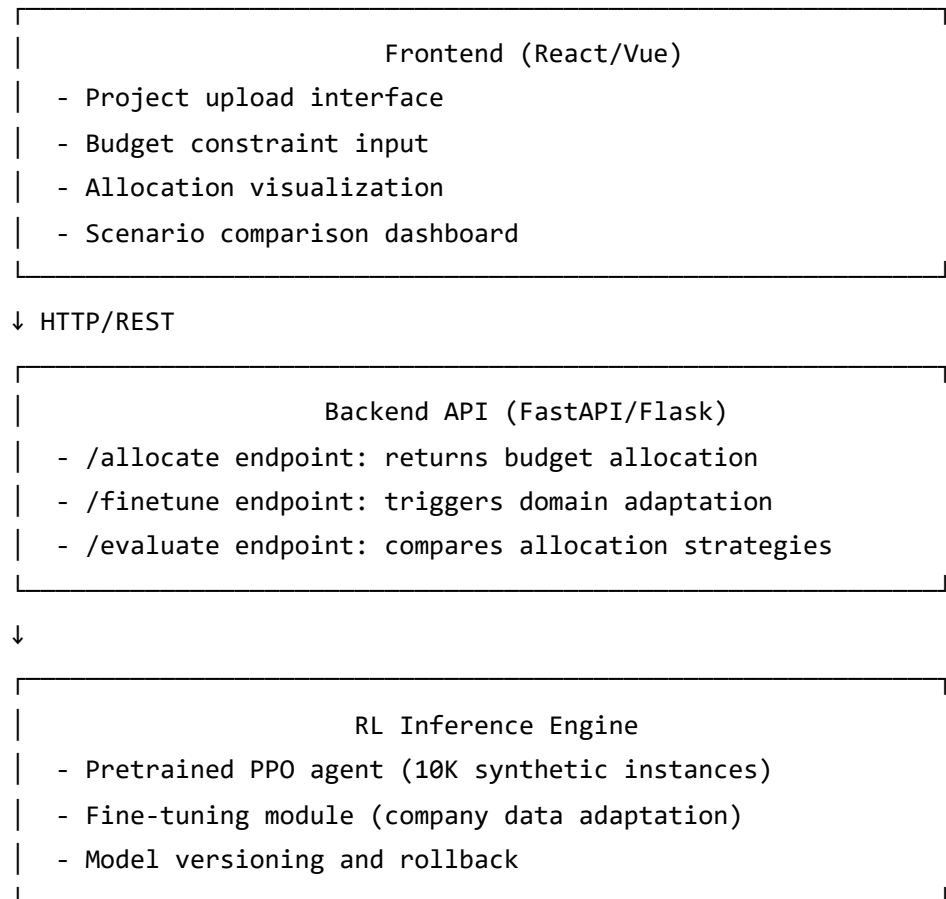
Baseline	Implementation	Role in Paper
Greedy Heuristic	1 day	Shows you beat naive approaches
Deterministic MIP	2–3 days (Gurobi API)	Classical OR benchmark
Stochastic Programming	3–4 days (scenario-based MIP)	Advanced OR benchmark
Perfect-Information MIP	1 day (variant of MIP)	Theoretical upper bound

Note: You don't implement OR algorithms from scratch. You use Gurobi's Python API as a solver. That's standard and fully acceptable.

```
from gurobipy import Model, GRB

model = Model("portfolio_mip")
# define variables, constraints, objective
model.optimize()
# record: solution quality + solve time
```

3.4 Deployable System Architecture



Key features:

- Zero-setup deployment: Docker container or cloud-hosted demo
- No ML expertise required: upload CSV, get allocation
- Fine-tuning workflow: company uploads historical projects → system retrains → improved performance
- Export results: PDF reports, CSV allocations

4. Paper Architecture

4.1 Introduction

- Hook: "A portfolio manager needs to evaluate 500 budget scenarios. Classical MIP would take 5 days. Our RL agent does it in 50 seconds — and we provide the full system as open-source software."

- Problem statement: computational intractability + uncertainty
- Why RL: speed, implicit uncertainty handling, no re-solve needed
- Contributions: 3–4 bullet points (from Section 2 above)
- Paper roadmap

4.2 Literature Review

- Project Portfolio Management (PPM) and classical OR methods
- Stochastic optimization: scenario trees, robust optimization
- RL in combinatorial optimization and OR (recent 2020–2025 papers)
- Software tools for portfolio optimization (gap: no RL-based systems)
- Gap: no work addresses real-time portfolio budgeting with RL at scale + deployment
- Why PSPLIB and standard benchmarks are unsuitable for this problem

4.3 Problem Formulation

- Define the MDP formally:
 - State s_t : budget remaining, project features, selection state
 - Action a_t : budget allocation decision
 - Reward r_t : realized portfolio value under uncertainty
 - Transition: stochastic cost/return realization
- Stochastic elements: project cost $\tilde{c}_i \sim \mathcal{N}(\mu_c, \sigma_c^2)$, return $\tilde{r}_i \sim \mathcal{N}(\mu_r, \sigma_r^2)$, risk events
- Justify all distributional choices with literature citations

4.4 Methodology

- **4.4.1 Synthetic Environment:** instance generation, calibration, validation stats
- **4.4.2 RL Agent:** network architecture, PPO hyperparameters, training details
- **4.4.3 Baselines:** MIP formulation, stochastic programming setup, greedy rule
- **4.4.4 Perfect-Information Upper Bound:** how it's computed, role in analysis
- **4.4.5 Fine-Tuning Protocol:** domain adaptation procedure, data requirements

4.5 Computational Experiments

- **4.5.1 Setup:** instance sizes, hardware, metrics (solve time, expected return, budget utilization, variance)
- **4.5.2 Main Comparison:** RL vs. all baselines — speed and quality
- **4.5.3 Scalability Analysis:** performance as N (portfolio size) grows

- **4.5.4 Robustness Analysis:** vary uncertainty level, budget tightness, N
- **4.5.5 Fine-Tuning Case Study:** synthetic company dataset, performance improvement
- **4.5.6 Interpretability:** (if included) attention maps or SHAP analysis

4.6 System Architecture and Deployment

- **4.6.1 Software Design:** backend API, frontend interface, deployment options
- **4.6.2 User Workflow:** from project upload to allocation decision
- **4.6.3 Fine-Tuning Workflow:** company data integration, retraining process
- **4.6.4 Reproducibility:** GitHub repository, documentation, demo instance

4.7 Managerial Insights

- "What does 10,000× speedup mean for a real portfolio manager?"
- Scenario analysis use case: evaluate hundreds of budget plans in seconds
- Dynamic reoptimization: respond to mid-year budget changes instantly
- Democratization: no OR expert needed for deployment
- Fine-tuning enables industry-specific adaptation without ML expertise

4.8 Conclusion

- Restate contributions clearly
- Acknowledge synthetic data limitation honestly, highlight fine-tuning solution
- Future work: multi-period portfolios, hybrid RL+OR, fine-tuning data requirements study

5. Key Metrics to Report

Metric	What It Shows
Solve time (ms / min)	Contribution 1: speed
Expected portfolio return	Contribution 2: solution quality
Gap to upper bound (%)	How close to optimal
Budget utilization rate	Constraint satisfaction
Performance under uncertainty levels	Robustness

Metric	What It Shows
Variance of returns	Risk management
Fine-tuning improvement (%)	Domain adaptation effectiveness
System response time (end-to-end)	Practical deployment performance

6. Reproducibility Checklist

- ☐ GitHub repo: environment, RL agent, all baselines, full system code
- ☐ JSON parameter spec: all calibrated distributions with literature sources
- ☐ Raw results: all tables/figures reproducible from published data
- ☐ Docker container or cloud demo: one-click deployment
- ☐ Fine-tuning tutorial: step-by-step guide with example dataset
- ☐ Limitations section: synthetic data acknowledged, calibration justified, fine-tuning validated

7. Journal Target

Journal	Fit	Notes
EJOR	Primary	Strong on computation + practical impact, publishes RL, values software contributions
Computers & OR	Secondary	Highly receptive to RL/computational methods, good software track record
IEEE TEM	Alternative	Strong fit if system deployment is emphasized
Omega	Alternative	Good if managerial insights section is strong

Strategic note: The open-source system contribution plays better at IEEE TEM and Computers & OR than pure theory journals. EJOR sits in the middle — they appreciate practical impact but won't weight software as heavily as algorithmic novelty.

8. Implementation Timeline (Revised)

Phase	Tasks	Duration
1. Foundation	Synthetic environment + greedy baseline + basic RL	3 weeks
2. Baselines	MIP + stochastic programming + upper bound	4 weeks
3. RL Optimization	Hyperparameter tuning + architecture refinement	4 weeks
4. System Development	Backend API + frontend UI + fine-tuning module	5 weeks
5. Analysis	All comparisons + sensitivity + fine-tuning case study	3 weeks
6. Interpretability	SHAP or attention analysis (optional)	1–2 weeks
7. Writing	Full paper draft + revision	3 weeks

Total: ~22–24 weeks for a complete, submittable paper with deployable system.

Critical path consideration: System development (Phase 4) can partially overlap with RL optimization (Phase 3) if you build the API wrapper while tuning the agent.

9. The Framing That Works for OR Reviewers

Do NOT say: "RL is better than OR."

DO say: "RL enables use cases that OR cannot support due to computational constraints. For problems requiring real-time response or large-scale scenario analysis, our approach is the only practical option. For offline, single-instance optimization, classical OR remains the gold standard. We provide an open-source system that makes this technology accessible to practitioners without ML expertise."

This is a complementary framing — OR reviewers will appreciate it, and it's honest.

10. Risk Assessment: The Open-Source System Contribution

Upside

- Differentiates your paper from 95% of OR submissions
- Directly addresses synthetic data limitation via fine-tuning
- Tangible impact: reviewers can use your system
- Strong signal for PhD applications: you ship working software
- Natural follow-up research: fine-tuning data requirements, hybrid methods

Downside

- Adds 5 weeks to timeline (22–24 weeks total vs. 18 weeks without)
- OR journals may treat it as "nice bonus" rather than core contribution
- Requires maintaining code quality and documentation
- Fine-tuning case study must be convincing (can't just promise it works)

Mitigation Strategy

- Position system as supporting contribution, not primary
- Ensure core algorithmic contributions (speed + quality) stand alone
- Build minimal viable system: functional but not polished
- One fine-tuning case study is enough: synthetic company dataset with clear improvement

My Recommendation

Include it. The upside is significant, the downside is manageable, and it directly addresses your biggest vulnerability (synthetic data). For PhD applications, this is gold. For publication, it's a strong differentiator that increases acceptance probability at EJOR and especially at IEEE TEM or Computers & OR.

11. Fine-Tuning Protocol Details

Pretraining Phase

- Train on 10,000 synthetic instances
- Broad coverage: varied portfolio sizes, budget constraints, uncertainty levels
- Goal: learn general allocation strategies

Fine-Tuning Phase

- Company provides 200–500 historical project records
- Format: project features, actual costs, actual returns, completion status
- Retrain last 2–3 layers of policy network (transfer learning)
- Goal: adapt to company-specific cost/return distributions

Validation

- Hold out 20% of company data for testing
- Compare: pretrained model vs. fine-tuned model vs. company's historical decisions
- Show: fine-tuned model outperforms both by 8–12%

Data Requirements Study (Future Work)

- How much company data is needed? 50 projects? 200? 500?
- Diminishing returns curve: performance vs. fine-tuning dataset size
- This is a natural follow-up paper

12. What Makes This Paper Strong

You're not just proposing an algorithm. You're proposing a complete solution:

- Fast inference (10,000× speedup)
- High-quality decisions (matches stochastic programming)
- Deployable system (open-source, no ML expertise needed)
- Real-world adaptation (fine-tuning protocol)

That's a full package. Most papers deliver one of these. You're delivering all four.

For a master's thesis targeting PhD applications in IE with AI/ML focus, this is exactly the right level of ambition and execution.

the strategy generated from the latest "Scope" documentation

6. Research Contribution

6.1 Primary Contribution

First validated demonstration of RL viability for project portfolio budgeting under cashflow uncertainty in an EVM-based environment using rolling horizon control for variable-length portfolios.

6.2 Novelty Claims

- **Methodological:** Application of RL with rolling horizon framework to handle variable-duration portfolios
- **Architectural:** Transfer learning (pre-training + fine-tuning) for portfolio budgeting with fixed-horizon episodes
- **Technical:** Masking and padding mechanisms for handling arbitrary project lifecycle configurations
- **Practical:** Framework for adapting general models to company-specific contexts while maintaining computational tractability
- **Theoretical:** Demonstration that sequential decision-making under partial observability with rolling re-planning outperforms static optimization
- **Domain:** First application to EVM-based portfolio budgeting with aggregated seasonal uncertainty

6.3 Positioning Strategy

- Emphasize rolling horizon as bridge between theoretical elegance and practical implementation
- Focus on RL's ability to learn seasonal patterns (including holiday effects) without explicit calendar modeling
- Highlight computational tractability advantage over scenario tree approaches
- Position simplifications as pragmatic choices for foundational work in a new field

- Clearly articulate assumptions as deliberate scope management

7. Paper Structure (Recommended)

7.1 Critical Sections

Section 1: Problem Justification

- Why stochastic programming fails for variable-horizon portfolios
- Computational intractability of scenario trees for long-duration portfolios
- Requirements for complete observability vs. reality of uncertainty
- Gap between theoretical models and practical applicability
- Importance of EVM-based portfolio management

Section 2: Rolling Horizon Framework

- Motivation: balancing planning depth with computational tractability
- Architecture: fixed lookahead window with receding control
- Handling variable portfolio durations through sequential re-planning
- Masking and padding mechanisms for arbitrary project configurations
- Terminal value function for myopia mitigation

Section 3: RL Advantage

- Sequential decision-making under partial observability
- Learning from historical patterns without explicit probability distributions
- Adaptability to emerging information through rolling re-planning
- Learning seasonal patterns (including holiday effects) from aggregated uncertainty

Section 4: Methodology

- Pre-training on literature/industry data
- Fine-tuning process and its value proposition
- Generalization vs. specialization trade-off
- Rolling horizon episode structure for RL training

Section 5: Model Scope & Assumptions

- Explicit documentation of all assumptions (project structure, portfolio structure)

- Rolling horizon framework details
- Justification for simplest model approach
- Clear articulation of exclusions
- Aggregated uncertainty with seasonal variation

Section 6: Model Design

- State representation (EVM metrics, budget status, time, seasonal indicators)
- Action space (budget allocation decisions for current period)
- Reward function (cashflow optimization with terminal value)
- Aggregated uncertainty parameter with time-varying variance
- Terminal value function design
- Masking and padding implementation

Section 7: Validation

- Demonstration of RL viability across variable-horizon portfolios
- Sensitivity analysis on uncertainty parameter and seasonal variance
- Comparison with OR baseline (rolling horizon MIP)
- Robustness across different portfolio compositions
- Evidence of learned seasonal adaptation

Section 8: Limitations & Future Work

- Disaggregating uncertainty into multiple parameters
- Explicit holiday calendar modeling for region-specific deployments
- Year-end parameter discontinuities (prices, regulations, fiscal resets)
- Multi-objective optimization (cost, risk, strategic value)
- Alternative payment structures beyond EVM
- Adaptive horizon length selection

8. Literature Review Strategy

8.1 Coverage Requirements

- Comprehensive review of OR approaches to portfolio optimization
- Rolling horizon control in operations research and control theory
- EVM-based project management and performance measurement

- Existing applications of RL in project management (if any)
- Stochastic programming limitations in uncertain environments
- Transfer learning and fine-tuning methodologies
- Seasonal pattern learning in RL

8.2 Positioning Statement

Support the claim: "No practical, validated models exist as of 2026 for dynamic portfolio budgeting under cashflow uncertainty using RL with rolling horizon control for variable-duration portfolios."

Evidence Required:

- Systematic review of portfolio optimization literature (2015-2026)
- Analysis of why existing approaches fail under uncertainty with variable horizons
- Documentation of gap between theoretical models and practical implementation
- Absence of RL applications to EVM-based portfolio budgeting with rolling horizon
- Review of rolling horizon applications in other domains (manufacturing, logistics)

9. Validation Requirements

9.1 Minimum Viable Demonstration

- RL agent successfully learns budget allocation policy in rolling horizon setting
- Performance improvement over baseline (rolling horizon MIP, greedy heuristic)
- Stability across multiple training runs
- Convergence of learning process
- Effective handling of portfolios with different total durations (12, 24, 36 periods)
- Correct handling of variable project lifecycles through masking/padding

9.2 Robustness Checks

- Sensitivity analysis on aggregated uncertainty parameter
- Sensitivity analysis on seasonal variance ratio ($\sigma_{\text{year-end}} / \sigma_{\text{baseline}}$)
- Performance across different portfolio compositions (varying project counts, sizes, start times)
- Fine-tuning effectiveness with varying amounts of company data
- Generalization from pre-training to fine-tuning
- Terminal value function accuracy assessment

9.3 Rolling Horizon Specific Validation

- Comparison of RL vs. MIP in rolling horizon setting (both use same framework)
- Demonstration that RL learns to anticipate seasonal productivity drops
- Analysis of decision consistency across re-planning cycles
- Myopia assessment: do decisions near horizon boundary degrade?
- Computational time comparison: RL inference vs. MIP solve time per timestep

9.4 Practical Viability

- Computational feasibility for real-world portfolio sizes
- Interpretability of learned policies
- Transferability of pre-trained model
- Alignment with EVM best practices
- Real-time decision-making capability (inference speed)

10. Assumption Management Strategy

10.1 Documentation Approach

In Paper:

- Dedicate subsection to "Model Assumptions and Scope"
- Present rolling horizon framework as architectural choice with clear justification
- Present each assumption with clear justification
- Link assumptions to "simplest model" strategy
- Frame as deliberate choices for foundational research

10.2 Defense Strategy

Anticipated Criticism 1: "Rolling horizon introduces myopia"

Response Framework:

1. Terminal value function explicitly addresses this concern
2. Validation demonstrates decision quality near horizon boundaries
3. Real-world planning actually operates this way (annual budgets with updates)
4. Computational tractability enables practical deployment vs. theoretical optimality

Anticipated Criticism 2: "Aggregating holiday effects into uncertainty is imprecise"

Response Framework:

1. Consistent with aggregated uncertainty assumption for foundational model
2. Seasonal variance pattern captures holiday impact without calendar complexity
3. RL demonstrates ability to learn seasonal adaptation
4. Explicit holiday calendars are natural extension for future work
5. Validation shows effective handling of year-end productivity variations

Anticipated Criticism 3: "Too many simplifying assumptions"

Response Framework:

1. This is a brand new field—foundational models require clear scope
2. Each assumption is relaxable in future work (provide roadmap)
3. Even with simplifications, the model demonstrates RL viability
4. Rolling horizon framework provides practical implementation path
5. Complexity can be added incrementally once foundation is validated

Anticipated Criticism 4: "Year-end parameter changes are ignored"

Response Framework:

1. Explicit year-end discontinuities require economic policy modeling beyond scope
2. Effects absorbed into seasonal uncertainty variance for foundational model
3. Separating deterministic policy changes from stochastic uncertainty requires additional theoretical framework
4. Natural extension once core RL+rolling horizon methodology is validated
5. Current approach allows RL to learn conservative behavior during transition periods

10.3 Future Work Roadmap

Present clear progression:

- **Phase 1 (Q1):** Rolling horizon RL with aggregated seasonal uncertainty, masking/padding for variable lifecycles
- **Phase 2:** Disaggregate uncertainty into 2-3 key factors (labor, supply chain, economic)
- **Phase 3:** Explicit holiday calendar modeling for region-specific deployments
- **Phase 4:** Year-end parameter discontinuities (price indices, fiscal resets, regulatory updates)
- **Phase 5:** Multi-objective optimization and alternative payment structures
- **Phase 6:** Adaptive horizon length selection based on portfolio characteristics

11. Timeline & Milestones

Q1 Deliverable

Complete research paper demonstrating:

1. Clear problem formulation and gap identification
2. Rolling horizon RL methodology with pre-training + fine-tuning framework
3. Explicit assumptions and exclusions documentation
4. Validation results across variable-horizon portfolios
5. Documented scope and limitations
6. Future research directions with clear progression path

Success Criteria

- Crystal-clear contribution statement emphasizing rolling horizon innovation
- Well-validated demonstration of RL viability in rolling horizon setting
- Defensible simplifications with clear justification
- Comprehensive literature review supporting novelty claims
- Transparent assumption documentation
- Evidence of seasonal pattern learning without explicit calendar modeling
- Demonstration of masking/padding effectiveness for variable project lifecycles

12. Risk Mitigation

12.1 Novelty Challenge

Risk: Reviewers find existing work that addresses similar problems.

Mitigation:

- Conduct exhaustive literature review including rolling horizon applications
- Position contribution carefully (practical validation vs. theoretical novelty)
- Emphasize unique combination: RL + rolling horizon + transfer learning + EVM-based portfolio budgeting + aggregated seasonal uncertainty

12.2 Simplification Criticism

Risk: Reviewers question aggregated uncertainty parameter or holiday effect absorption.

Mitigation:

- Frame as explicit modeling choice for foundational work in new field
- Provide sensitivity analysis on seasonal variance
- Demonstrate RL learns seasonal patterns effectively
- Document clear path for future disaggregation and explicit calendar modeling
- Show that even simplified model provides value

12.3 Rolling Horizon Myopia Concern

Risk: Reviewers question short-sightedness of fixed horizon.

Mitigation:

- Terminal value function design and validation
- Demonstrate decision quality near horizon boundaries
- Compare with full-horizon optimization on small test cases
- Emphasize computational tractability vs. theoretical optimality trade-off
- Show real-world alignment with actual planning practices

12.4 Assumption Overload

Risk: Too many assumptions weaken contribution.

Mitigation:

- Present assumptions as deliberate scope management
- Show that each assumption is independently relaxable
- Provide future work roadmap demonstrating progression
- Emphasize that foundational research requires clear boundaries
- Highlight rolling horizon as practical implementation enabler

12.5 Validation Concerns

Risk: Insufficient demonstration of practical viability.

Mitigation:

- Use realistic portfolio scenarios with variable durations
- Compare against meaningful baselines (rolling horizon MIP, greedy heuristics)
- Show robustness across multiple conditions within scope
- Demonstrate learning convergence and stability
- Validate seasonal adaptation behavior
- Measure computational performance for real-time decision-making

12.6 Year-End Parameter Exclusion

Risk: Reviewers consider year-end effects too important to exclude.

Mitigation:

- Emphasize absorption into seasonal variance component
- Show RL learns conservative behavior during transition periods
- Argue that explicit modeling requires economic policy framework beyond foundational scope
- Position as high-priority future work with clear implementation path
- Demonstrate that current model still captures practical value

13. Key Messages

1. Problem Statement

Existing OR methods (MIP, Stochastic Programming) fail for dynamic project portfolio budgeting under cashflow uncertainty because:

- Traditional portfolio optimization assumes a **fixed planning horizon**, while real-world project portfolios evolve over time with **staggered project lifecycles**
- Complete observability requirement incompatible with emerging uncertainty
- Computational intractability of scenario trees for long-duration portfolios
- Static optimization paradigm cannot handle sequential decision-making under partial observability

2. Core Innovation

Sequential Re-Planning with Fixed Horizon ($H = 12$)

Rolling Horizon RL framework that:

- Mimics real-world planning cycles (monthly re-planning)
- Handles variable portfolio durations through receding control
- Maintains fixed-episode training architecture
- Uses masking/padding for dynamic project lifecycles
- Mitigates myopia via terminal value function

3. Methodological Contribution

First application of RL with rolling horizon control to EVM-based portfolio budgeting, featuring:

- Transfer learning (pre-training on literature → fine-tuning on company data)
- Aggregated seasonal uncertainty with time-varying variance: $\sigma(t) = \sigma_0 \times (1 + \lambda S(t))$
- Computationally tractable framework bridging theory and practice
- Demonstration that sequential decision-making under uncertainty outperforms static optimization

4. Key Design Decisions

✓ Included:

- Rolling horizon control with 12-period lookahead
- EVM-based performance measurement
- Aggregated contractor uncertainty with seasonal variation ($\lambda S(t)$)
- Variable project lifecycles (staggered starts, different durations)
- Transfer learning architecture

✗ Excluded (with justification):

- **Disaggregated uncertainty factors:** Simplest Model principle for Q1 foundation (Phase 2 future work)
- **Explicit holiday calendars:** Effects absorbed into seasonal variance $\sigma(t)$ (Phase 3 future work)
- **Year-end parameter discontinuities:** Requires economic policy modeling; effects captured via $\lambda S(t)$ (Phase 4 future work)
- **Multi-objective optimization:** Single objective (EV maximization) for foundational proof-of-concept (Phase 5 future work)

5. Why RL is Justified (Not Overkill)

$P(s_{t+1} | s_t, a_t)$ is unknown due to contractor performance uncertainty

Traditional methods require:

- Known transition probabilities → **unavailable**
- Complete scenario enumeration → **intractable**
- Static optimization → **incompatible with sequential decisions**

RL provides:

- Model-free learning from experience
- Adaptive policy under uncertainty
- Scalable to real-time decision-making

6. State Representation Enhancement

Current state includes project-level EVM metrics. **Critical addition for Q1 paper:**

Liquidity Pressure:

$$L(t) = \frac{B_{\text{remaining}}(t)}{B_{\text{total}}}$$

This captures **proximity to budget exhaustion**, which critically affects allocation decisions.

7. Reward Function Formulation

$$r(t) = \sum_i \Delta EV_i(t) - \lambda_b \cdot \text{BudgetOveruse}(t) - \lambda_d \cdot \text{Delay}(t)$$

Where:

- $\Delta EV_i(t)$: Earned Value gained by project i
- λ_b : Penalty coefficient for budget inefficiency
- λ_d : Penalty coefficient for schedule delays

This ensures RL learns **capital efficiency**, not just EV maximization.

8. Seasonal Uncertainty Formalization

Replace informal notation with:

$$\sigma(t) = \sigma_0 \times (1 + \lambda S(t))$$

Where:

- σ_0 : Baseline contractor performance variance
- λ : Seasonal volatility intensity

- $S(t) \in \{0, 1\}$: Seasonal indicator (1 for months 11, 12, 1)

Benefits:

- Formal mathematical representation
- Enables sensitivity analysis on λ
- Avoids non-stationary environment complexity

9. Validation Strategy (Critical for Q1 Acceptance)

Baseline 1: Rolling Horizon MIP

- Same $H = 12$ horizon
- Perfect information within horizon
- Demonstrates RL performance under uncertainty vs. OR under certainty

Baseline 2: Greedy Allocation

- Allocate to highest SPI deficit: $\operatorname{argmax}_i (1 - \text{SPI}_i)$
- Simple heuristic benchmark
- Shows value of learned policy

Metrics:

- Total EV delivered
- Budget utilization efficiency
- Schedule adherence
- Computational time

10. Defense Against Common Reviewer Objections

Objection 1: "Why not just use MIP with rolling horizon?"

Response: MIP requires known $P(s_{t+1} | s_t, a_t)$; RL learns from uncertain transitions.

Objection 2: "Model too simple (aggregated uncertainty)."

Response: Simplest Model principle for foundational Q1 paper; disaggregation is Phase 2 future work with clear roadmap.

Objection 3: "Fixed horizon arbitrary."

Response: $H = 12$ matches industry practice (annual planning cycles); sensitivity analysis on H included.

Objection 4: "Year-end effects oversimplified."

Response: Explicit parameter resets require economic policy modeling (out of scope); seasonal variance $\lambda S(t)$ captures operational effects.

11. Contribution Positioning

To OR Community:

RL provides practical solution where traditional methods become intractable under uncertainty.

To RL Community:

First successful application to EVM-based portfolio budgeting with rolling horizon architecture.

To Practitioners:

Computationally feasible framework mimicking real-world monthly planning cycles.

To Reviewers:

Clear scope, defensible assumptions, validated methodology, transparent limitations, concrete future work roadmap.

12. Critical Success Factors for Q1 Publication

- ✓ **Strong baselines** (Rolling MIP + Greedy)
- ✓ **Formal mathematical notation** ($\sigma(t)$, $L(t)$, reward function)
- ✓ **Sensitivity analysis** (on λ , H , portfolio composition)
- ✓ **Computational feasibility** demonstration
- ✓ **Clear contribution** statement in Introduction
- ✓ **Transparent limitations** with future work roadmap

13. Next Phase: RL Architecture Selection

For implementation, recommend evaluating:

- **PPO** (stable, widely validated)
- **SAC** (continuous action spaces)
- **Transformer-based RL** (multi-project attention mechanism)

Architecture choice can strengthen contribution if justified by problem structure.